

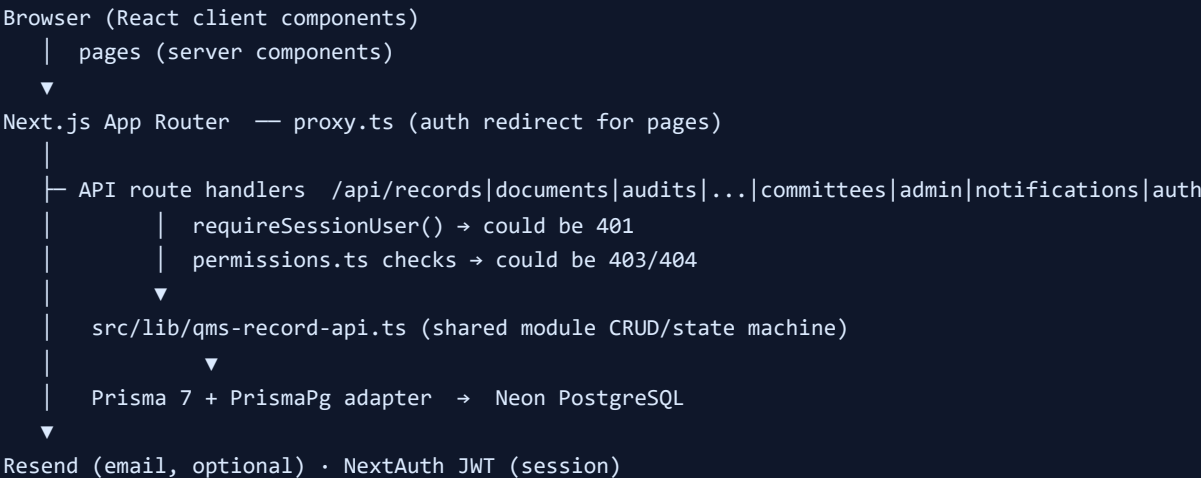
TECHNICAL REQUIREMENTS DOCUMENT

QMS — System Design & Architecture

Version 1.0 | Sep 2026 | Stack: Next.js 16 · React 19 · TypeScript strict · Prisma 7 · Neon PostgreSQL · NextAuth v4 · Resend

1. Architecture Overview

QMS is a single Next.js application using the **App Router**. Page routes are server components; record detail/create pages delegate logic to one shared, module-parameterized implementation. A `proxy.ts` (Next 16 middleware-equivalent) guards page navigation; every `/api/*` route independently authenticates via NextAuth and authorizes via the permissions module.



2. Repository Layout

Path	Responsibility
<code>app/[module]/...</code> , <code>app/committees/...</code> , <code>app/admin/...</code> , etc.	Page routes (list/new/detail/edit), dashboards, admin screens
<code>app/api/...</code>	REST route handlers per module, records, committees, admin, notifications, auth
<code>src/lib/qms-record-api.ts</code>	Shared record API – CRUD, state transitions, history/comments, module factory
<code>src/lib/module-route-factory.ts</code>	Per-module route wiring on the shared API
<code>src/lib/permissions.ts</code> / <code>lib/permissions.ts</code>	Authorization engine (server truth) + client-safe re-export
<code>src/lib/api-auth.ts</code>	<code>requireSessionUser()</code> – session user + full profile list
<code>src/lib/auth.ts</code> / <code>lib/auth.ts</code>	NextAuth options (credentials, JWT) and re-export

<code>src/lib/email.ts</code>	Resend client, email templates, in-app notification creation
<code>src/lib/prisma.ts</code> / <code>lib/prisma.ts</code>	Prisma client (PrismaPg adapter), singleton; re-export
<code>lib/qms-cache.ts</code>	Client-side cached profile/record fetch layer
<code>lib/qms-data.ts</code>	Seeded org tree, records, committees, memberships, tasks
<code>components/</code>	Shared UI: module core (<code>qms.tsx</code>), dashboard, approval panel, committees, profile display
<code>prisma/</code>	Schema, migrations, seed script
<code>proxy.ts</code>	Next proxy – boot-page auth gate

3. Environment & Configuration

Variable	Required	Purpose
<code>DATABASE_URL</code>	Yes	PostgreSQL connection (PrismaPg reads it directly)
<code>NEXTAUTH_URL</code>	Yes (prod)	Canonical app URL for callbacks
<code>NEXTAUTH_SECRET</code>	Yes	JWT signing secret (32+ bytes)
<code>RESEND_API_KEY</code>	No	Enables transactional email (<code>isEmailEnabled()</code>)
<code>RESEND_FROM_EMAIL</code>	No	Sender; default <code>QMS Notifications <notifications@qms.local></code>

4. Data Model (Prisma)

Key relationships (see Backend Schema doc for full field list):

```

Profile └─ reportsTo → Profile (self-relation, org tree)
      │ documentAssignments / capaAssignments / ...
      │ committeeCreations, committeeMemberships
      │ committeeTaskAssignments / AssignedBy
      └─ passwordResetTokens

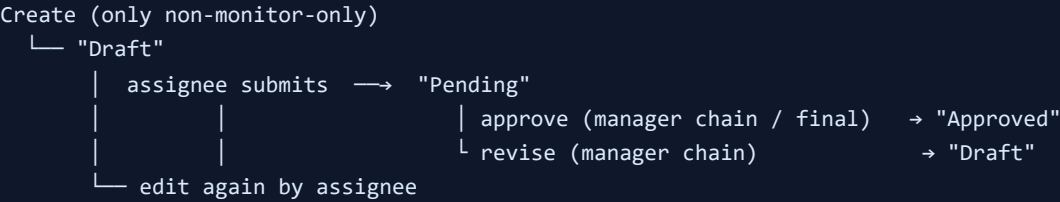
Record modules (Document, Capa, Nonconformance, Audit, Training)
  └─ all: id, assignedTo → Profile, status, deletedAt?

Committee — CommitteeMembership (profileId, roleInCommittee head|member)
  └─ CommitteeTask (assignedTo, assignedBy, status assigned|in_progress|submitted|approved)

RecordHistory (recordType, recordId, fromStatus, toStatus, changedBy, comment)
Comment (recordType, recordId, authorId, body)
Notification (userId, message, link, read)
PasswordResetToken (profileId, tokenHash, expiresAt, usedAt)

```

5. Record State Machine



Transitions persist in `RecordHistory` ; statuses are stored per module and render via `StatusBadge` styling map in `components/qms.tsx` .

6. Authorization Engine

Function	Rule
<code>isMonitorOnly</code>	<code>profile.reportsTo === null</code> (top authority)
<code>canCreateRecords</code> / <code>canAssignRecords</code>	NOT monitor-only
<code>canViewRecord</code>	<code>viewer === assignee</code> OR <code>viewer ∈ assignee's superior chain</code> (scope = self + subordinates)
<code>canSubmitOrUpdate</code>	<code>viewer === assignee</code>
<code>canApproveOrRevise</code>	walk <code>assignee.reportsTo</code> chain upward; viewer must appear in it
<code>canManageCommittees</code>	NOT monitor-only AND profile flag <code>canManageCommittees</code>
<code>canAssignCommitteeTask</code>	committee manager OR committee head (per membership)
<code>canGrantCommitteePermission</code>	top authority only
<code>canInitiateSuccession</code>	top authority only

7. API Surface (abridged)

Method + Path	Behavior
POST/GET <code>/api/{module}</code> · <code>/api/records</code>	Create (201 with guard) / list within scope
GET/PATCH <code>/api/{module}/{id}</code>	Detail (404 if deleted), update (assignee only)
POST <code>/api/{module}/{id}/submit approve revise</code>	State transitions with permission checks
GET/POST <code>/api/{module}/{id}/comments</code>	Comment feed
POST <code>/api/records/assign</code>	Assign record to a subordinate; emits notification + optional email

GET/POST/DELETE <code>/api/committees...</code>	Committee CRUD, members, tasks
GET/PATCH <code>/api/admin/team</code>	List profiles; edit email / add user
POST <code>/api/admin/permissions</code> / <code>/api/admin/succession</code> / <code>/api/admin/test-email</code>	Admin tools (top authority)
GET <code>/api/notifications</code>	Current user's notifications
POST <code>/api/auth/...</code>	NextAuth credentials flow

8. Security Requirements

- AUTH-1 Do not expose `passwordHash` in any API response (`toPublicProfile` strips it).
- AUTH-2 Verify session on every mutating API; respond 401/403 rather than falling through.
- AUTH-3 Reset tokens stored hashed, expiring, single-use.
- AUTH-4 Secret values only from environment; never commit `.env*`.
- AUTH-5 Email is best-effort: a Resend failure must not roll back the DB transaction.

9. Deployment & Operations

Concern	Detail
Build	<code>postinstall: prisma generate --config prisma.config.ts</code> then <code>next build</code>
Runtime	Neon PostgreSQL; pooler connection string; <code>channel_binding=require</code>
Host	Vercel (functions), env vars per environment (Production/Preview/Development)
Migrations	Prisma migrate (6 migrations tracked in <code>prisma/migrations</code>)
Verification	<code>npx tsc --noEmit</code> · <code>npx eslint .</code> · <code>npm run build</code> · <code>npm run prisma:seed</code> (destructive)